

Prompt para generar la interfaz en Qt Designe

Actúa como experto en diseño de interfaces con Qt Designer para aplicaciones desarrolladas en Python

y PySide6.

Necesito diseñar una sección para administrar y representar un grafo no dirigido dentro de una

aplicación académica de estructuras de datos.

La aplicación ya contiene otras estructuras, como pilas, colas, listas y árboles. La nueva interfaz

debe poder integrarse como una página de un QStackedWidget, como una pestaña o como una sección de la

ventana principal.

No debes generar código Python. Debes proporcionar instrucciones precisas para construir la interfaz

directamente en Qt Designer.

Universidad de León – Programación II – Ing. Alejandro Montes

Práctica de grafos no dirigidos | 7

La interfaz debe incluir los siguientes componentes y utilizar exactamente los objectName

especificados.

1. Título principal

Agregar un QLabel con el texto: "Grafos no dirigidos".

2. Sección "Vértices"

Agregar un QGroupBox con el título "Vértices". Dentro colocar:

- QLineEdit llamado txtVertex, con placeholderText "Nombre del vértice".

- QPushButton llamado btnAddVertex, texto "Agregar vértice".
- QComboBox llamado cmbVertex.
- QPushButton llamado btnDeleteVertex, texto "Eliminar vértice".

3. Sección "Arcos"

Agregar un QGroupBox con el título "Arcos". Dentro colocar:

- QLabel con el texto "Vértice 1".
- QComboBox llamado cmbOrigin.
- QLabel con el texto "Vértice 2".
- QComboBox llamado cmbDestination.
- QPushButton llamado btnAddEdge, texto "Agregar arco".
- QPushButton llamado btnDeleteEdge, texto "Eliminar arco".

4. Sección de representaciones

Agregar un QTabWidget que ocupe la mayor parte del espacio disponible y contenga cuatro pestañas.

Primera pestaña: "Representación visual".

- QGraphicsView llamado graphicsViewGraph.
- Debajo, QPushButton llamado btnRedrawGraph, texto "Redibujar grafo".
- Organizar con QVBoxLayout.
- No dibujar manualmente círculos, líneas o imágenes; el dibujo se generará dinámicamente desde

Python.

Segunda pestaña: "Matriz de adyacencia".

- QTableWidgetItem llamado tblAdjacencyMatrix.
- Solo lectura, con encabezados horizontales y verticales.

Tercera pestaña: "Lista de adyacencia".

- QPlainTextEdit llamado txtAdjacencyList.
- Solo lectura.

Cuarta pestaña: "Lista de arcos".

- QTableWidgetItem llamado tblEdges.
- Dos columnas con encabezados "Vértice 1" y "Vértice 2".
- Solo lectura.

5. Parte inferior

Agregar:

- QLabel llamado lblGraphStatus, texto inicial "Vértices: 0 | Arcos: 0".
- QPushButton llamado btnClearGraph, texto "Limpiar grafo".
- Organizar ambos componentes en un QHBoxLayout.

6. Condiciones generales

- Utilizar layouts en todos los niveles.

Universidad de León – Programación II – Ing. Alejandro Montes

Práctica de grafos no dirigidos | 8

- No utilizar posiciones absolutas.
- Organizar las secciones de captura mediante QGroupBox.
- Mantener una distribución clara y académica.
- Permitir redimensionar la ventana.
- Dar mayor espacio al QTabWidget.
- Permitir que graphicsViewGraph se redimensione junto con la ventana.
- Usar exactamente los objectName indicados.
- No cambiar ni traducir los objectName.
- No agregar controles para dirección, pesos o tipos de grafo.
- La interfaz representará exclusivamente un grafo no dirigido.

- El QGraphicsView será utilizado posteriormente desde Python para dibujar los vértices mediante

círculos y los arcos mediante líneas.

- Todos los componentes deben permanecer dentro de layouts.

- La interfaz debe quedar preparada para conectarse con una clase Graph.

Explica paso a paso qué componente colocar, en qué orden, qué layout aplicar, qué propiedades

configurar, qué objectName asignar, cómo configurar los componentes como solo lectura, cómo hacer que

el QGraphicsView ocupe el espacio disponible, cómo verificar que todos los elementos estén dentro de

layouts y cómo guardar el archivo .ui para integrarlo con Python.

Instrucciones paso a paso para diseñar la interfaz en Qt Designer

Como experto en diseño de interfaces con PyQt/PySide y Qt Designer, te guiaré de forma estructurada para construir esta sección de grafos no dirigidos, asegurando una distribución académica limpia, flexible y totalmente basada en layouts (sin posiciones absolutas).

Paso 1: Crear el contenedor principal de la sección

Para permitir que esta sección se integre fácilmente como una pestaña, una página de un QStackedWidget o un contenedor principal:

1. Abre Qt Designer y crea un nuevo widget base (puedes seleccionar **Widget** o **Main Window** según convenga a tu proyecto; si es para un QStackedWidget, un **Widget** vacío es ideal).
2. Arrastra un **Vertical Layout** (QVBoxLayout) principal que cubra todo el contenedor base. Esto asegurará que todos los elementos se adapten al redimensionar la ventana.

Paso 2: Crear el Título principal

1. Arrastra un **QLabel** dentro del layout principal vertical.
2. Modifica su propiedad de texto a: Grafos no dirigidos.

3. (Opcional) Ajusta el tamaño de fuente en sus propiedades para que destaque como encabezado principal.

Paso 3: Crear las secciones de captura superiores (Vértices y Arcos)

Para organizar los controles de entrada de manera ordenada y simétrica, utilizaremos un layout horizontal principal que contenga a los dos QGroupBox.

1. Arrastra un **Horizontal Layout** (QHBoxLayout) y colócalo debajo del título principal.

3.1. Sección "Vértices"

1. Arrastra un **QGroupBox** dentro del layout horizontal anterior.
2. Configura su propiedad title como: Vértices.
3. Aplica un **Vertical Layout** o **Grid Layout** dentro de este QGroupBox.
4. Coloca los siguientes componentes en orden:
 - **QLineEdit**: Asigna el objectName **txtVertex** y configura su propiedad placeholderText como Nombre del vértice.
 - **QPushButton**: Asigna el objectName **btnAddVertex** y establece el texto en Agregar vértice.
 - **QComboBox**: Asigna el objectName **cmbVertex**.
 - **QPushButton**: Asigna el objectName **btnDeleteVertex** y establece el texto en Eliminar vértice.

3.2. Sección "Arcos"

1. Arrastra un segundo **QGroupBox** dentro del layout horizontal principal (junto al de vértices).
2. Configura su propiedad title como: Arcos.
3. Aplica un **Grid Layout** (cuadrícula) dentro de este QGroupBox para ordenar las etiquetas y los selectores de origen/destino de forma limpia:
 - Fila 0: Un **QLabel** con el texto Vértice 1 y al lado el **QComboBox** con objectName **cmbOrigin**.
 - Fila 1: Un **QLabel** con el texto Vértice 2 y al lado el **QComboBox** con objectName **cmbDestination**.

- Fila 2: Un **QPushButton** con objectName **btnAddEdge** y texto Agregar arco.
- Fila 3: Un **QPushButton** con objectName **btnDeleteEdge** y texto Eliminar arco.

Paso 4: Crear la sección de representaciones (QTabWidget)

Para cumplir con el requisito de que esta sección ocupe la mayor parte del espacio disponible y se redimensione dinámicamente:

1. Arrastra un **Tab Widget** (QTabWidget) y colócalo en el layout principal vertical, **debajo** de la sección de vértices y arcos.
2. Configuración de expansión: Selecciona el QTabWidget, busca la propiedad sizePolicy y asegúrate de que el crecimiento vertical (Vertical Policy) esté configurado en **Expanding** (Expandible). Esto le dará prioridad de espacio frente a los controles superiores.

Pestaña 1: "Representación visual"

1. Haz clic derecho sobre las pestañas del QTabWidget para renombrar o configurar la primera pestaña (o haz clic en ella). Su etiqueta por defecto cambiará a Representación visual.
2. Aplica un **Vertical Layout** (QVBoxLayout) dentro de esta pestaña.
3. Arrastra un **QGraphicsView** dentro de este layout y asígnale el objectName **graphicsViewGraph**.
 - Nota de diseño: Su sizePolicy debe configurarse en **Expanding** tanto en horizontal como en vertical para que el área gráfica crezca junto con la ventana.
4. Debajo del QGraphicsView, arrastra un **QPushButton**, asígnale el objectName **btnRedrawGraph** y establece su texto en Redibujar grafo.

Pestaña 2: "Matriz de adyacencia"

1. Añade una segunda pestaña al QTabWidget y nómbrala Matriz de adyacencia.
2. Aplica un layout vertical en su interior.
3. Arrastra un **Table Widget** (QTableWidget), asígnale el objectName **tblAdjacencyMatrix**.

4. Configuración de solo lectura y encabezados: En las propiedades del `QTableWidget`, busca la opción `editTriggers` y desmarca todas (o selecciona `NoEditTriggers`) para garantizar que sea de **solo lectura**. Asegúrate de que las opciones de visibilidad de encabezados horizontales y verticales (`horizontalHeaderVisible` y `verticalHeaderVisible`) estén marcadas como verdaderas.

Pestaña 3: "Lista de adyacencia"

1. Añade una tercera pestaña y nómbrala Lista de adyacencia.
2. Aplica un layout vertical en su interior.
3. Arrastra un **Plain Text Edit** (`QPlainTextEdit`), asígnale el `objectName` **txtAdjacencyList**.
4. Configuración de solo lectura: En sus propiedades, marca la propiedad `readOnly` como `true`.

Pestaña 4: "Lista de arcos"

1. Añade la cuarta pestaña y nómbrala Lista de arcos.
2. Aplica un layout vertical en su interior.
3. Arrastra un **Table Widget** (`QTableWidget`), asígnale el `objectName` **tblEdges**.
4. Configuración de columnas y solo lectura: Configura sus propiedades de edición como `NoEditTriggers` (solo lectura). Mediante las propiedades de la tabla o vía código en Python, configurala con **2 columnas** cuyos encabezados sean "Vértice 1" y "Vértice 2".

Paso 5: Crear la parte inferior (Estado y Limpieza)

1. Arrastra un **Horizontal Layout** (`QHBoxLayout`) y colócalo en la parte inferior del layout principal vertical.
2. Dentro de este layout horizontal, añade:
 - Un **QLabel** con el `objectName` **lblGraphStatus** y texto inicial Vértices: 0 | Arcos: 0.
 - Un **Spacer** horizontal (`Horizontal Spacer`) para empujar los elementos y mantener una distribución limpia.

- Un **QPushButton** con el objectName **btnClearGraph** y texto Limpiar grafo.

Paso 6: Verificaciones de diseño e integridad estructural

Antes de dar por concluida la interfaz, realiza las siguientes verificaciones en Qt Designer:

1. **Verificar layouts en todos los niveles:** Selecciona el widget raíz de la ventana o contenedor y haz clic en el botón **Lay Out Vertically** o **Lay Out in a Grid** en la barra de herramientas superior de Qt Designer. Ningún elemento debe flotar libremente sin un layout asignado.
2. **Prueba de redimensionamiento:** Presiona las teclas Ctrl + R (Vista previa en Qt Designer) y redimensiona la ventana emergente. Comprueba que el QTabWidget y el graphicsViewGraph se expandan correctamente mientras los botones y campos conservan proporciones estables.
3. **Validación estricta de nombres (objectName):** Comprueba en el panel derecho (Object Inspector) que los nombres coincidan exactamente con los requeridos:
 - txtVertex
 - btnAddVertex
 - cmbVertex
 - btnDeleteVertex
 - cmbOrigin
 - cmbDestination
 - btnAddEdge
 - btnDeleteEdge
 - graphicsViewGraph
 - btnRedrawGraph
 - tblAdjacencyMatrix
 - txtAdjacencyList
 - tblEdges

- lblGraphStatus
- btnClearGraph

Paso 7: Guardar el archivo para su integración con Python

1. Ve al menú superior **File > Save As...** (Guardar como...).
2. Guarda el archivo con el nombre exacto requerido por tu arquitectura de proyecto dentro de la carpeta correspondiente (por ejemplo, graph_view.ui o integrado en los recursos de tu interfaz académica).
3. Este archivo .ui quedará listo para ser cargado en tu aplicación principal de Python mediante QUiLoader o loadUi de PySide6, permitiéndote conectar de inmediato las señales de los botones y manipular las estructuras de datos del grafo no dirigido.

Muy bien, ahora dame el código completo de ui por favor, llamado grafos.ui

XML

```
<?xml version="1.0" encoding="UTF-8"?>

<ui version="4.0">

<class>GrafosWindow</class>

<widget class="QWidget" name="GrafosWindow">

<property name="geometry">

<rect>

<x>0</x>

<y>0</y>

<width>1000</width>

<height>750</height>

</rect>

</property>

<property name="windowTitle">

<string>Grafos no dirigidos</string>

</property>
```

```
<layout class="QVBoxLayout" name="verticalLayout_main">
  <property name="spacing">
    <number>12</number>
  </property>
  <property name="leftMargin">
    <number>16</number>
  </property>
  <property name="topMargin">
    <number>16</number>
  </property>
  <property name="rightMargin">
    <number>16</number>
  </property>
  <property name="bottomMargin">
    <number>16</number>
  </property>
  <item>
    <widget class="QLabel" name="label_title">
      <property name="font">
        <font>
          <pointsize>14</pointsize>
          <bold>true</bold>
        </font>
      </property>
      <property name="text">
        <string>Grafos no dirigidos</string>
      </property>
    </widget>
  </item>
</layout>
```

```
</property>
</widget>
</item>
<item>
<layout class="QHBoxLayout" name="horizontalLayout_controls">
<property name="spacing">
<number>12</number>
</property>
<item>
<widget class="QGroupBox" name="groupBox_vertices">
<property name="title">
<string>Vértices</string>
</property>
<layout class="QVBoxLayout" name="verticalLayout_vertices">
<item>
<widget class="QLineEdit" name="txtVertex">
<property name="placeholderText">
<string>Nombre del vértice</string>
</property>
</widget>
</item>
<item>
<widget class="QPushButton" name="btnAddVertex">
<property name="text">
<string>Agregar vértice</string>
</property>
```

```
</widget>

</item>

<item>

  <widget class="QComboBox" name="cmbVertex"/>

</item>

<item>

  <widget class="QPushButton" name="btnDeleteVertex">

    <property name="text">

      <string>Eliminar vértice</string>

    </property>

  </widget>

</item>

</layout>

</widget>

</item>

<item>

  <widget class="QGroupBox" name="groupBox_arcos">

    <property name="title">

      <string>Arcos</string>

    </property>

    <layout class="QGridLayout" name="gridLayout_arcos">

      <item row="0" column="0">

        <widget class="QLabel" name="label_origin">

          <property name="text">

            <string>Vértice 1</string>

          </property>
```

```
</widget>

</item>

<item row="0" column="1">

  <widget class="QComboBox" name="cmbOrigin"/>

</item>

<item row="1" column="0">

  <widget class="QLabel" name="label_destination">

    <property name="text">

      <string>Vértice 2</string>

    </property>

  </widget>

</item>

<item row="1" column="1">

  <widget class="QComboBox" name="cmbDestination"/>

</item>

<item row="2" column="0" colspan="2">

  <widget class="QPushButton" name="btnAddEdge">

    <property name="text">

      <string>Agregar arco</string>

    </property>

  </widget>

</item>

<item row="3" column="0" colspan="2">

  <widget class="QPushButton" name="btnDeleteEdge">

    <property name="text">

      <string>Eliminar arco</string>
```

```
</property>
</widget>
</item>
</layout>
</widget>
</item>
</layout>
</item>
<item>
<widget class="QTabWidget" name="tabWidget_representations">
<property name="sizePolicy">
<sizepolicy hsize="Expanding" vsize="Expanding">
<horstretch>0</horstretch>
<verstretch>1</verstretch>
</sizepolicy>
</property>
<property name="currentIndex">
<number>0</number>
</property>
<widget class="QWidget" name="tab_visual">
<attribute name="title">
<string>Representación visual</string>
</attribute>
<layout class="QVBoxLayout" name="verticalLayout_visual">
<item>
<widget class="QGraphicsView" name="graphicsViewGraph">
```

```
<property name="sizePolicy">
    <sizepolicy hsize="Expanding" vsize="Expanding">
        <horstretch>0</horstretch>
        <verstretch>0</verstretch>
    </sizepolicy>
</property>
</widget>
</item>
<item>
    <widget class="QPushButton" name="btnRedrawGraph">
        <property name="text">
            <string>Redibujar grafo</string>
        </property>
    </widget>
</item>
</layout>
</widget>
<widget class="QWidget" name="tab_matrix">
    <attribute name="title">
        <string>Matriz de adyacencia</string>
    </attribute>
    <layout class="QVBoxLayout" name="verticalLayout_matrix">
        <item>
            <widget class="QTableWidget" name="tblAdjacencyMatrix">
                <property name="editTriggers">
                    <set>QAbstractItemView::NoEditTriggers</set>
```

```
</property>
</widget>
</item>
</layout>
</widget>
<widget class="QWidget" name="tab_adjacency_list">
  <attribute name="title">
    <string>Lista de adyacencia</string>
  </attribute>
  <layout class="QVBoxLayout" name="verticalLayout_adj_list">
    <item>
      <widget class="QPlainTextEdit" name="txtAdjacencyList">
        <property name="readOnly">
          <bool>true</bool>
        </property>
      </widget>
    </item>
  </layout>
</widget>
<widget class="QWidget" name="tab_edges_list">
  <attribute name="title">
    <string>Lista de arcos</string>
  </attribute>
  <layout class="QVBoxLayout" name="verticalLayout_edges_list">
    <item>
      <widget class="QTableWidget" name="tblEdges">
```

```
<property name="editTriggers">
  <set>QAbstractItemView::NoEditTriggers</set>
</property>
<column>
  <property name="text">
    <string>Vértice 1</string>
  </property>
</column>
<column>
  <property name="text">
    <string>Vértice 2</string>
  </property>
</column>
</widget>
</item>
</layout>
</widget>
</widget>
</item>
<item>
  <layout class="QHBoxLayout" name="horizontalLayout_footer">
    <item>
      <widget class="QLabel" name="lblGraphStatus">
        <property name="text">
          <string>Vértices: 0 | Arcos: 0</string>
        </property>
```

```
</widget>
</item>
<item>
  <spacer name="horizontalSpacer">
    <property name="orientation">
      <enum>Qt::Horizontal</enum>
    </property>
    <property name="sizeHint" stdset="0">
      <size>
        <width>40</width>
        <height>20</height>
      </size>
    </property>
  </spacer>
</item>
<item>
  <widget class="QPushButton" name="btnClearGraph">
    <property name="text">
      <string>Limpiar grafo</string>
    </property>
  </widget>
</item>
</layout>
</item>
</layout>
</widget>
```

<resources/>

<connections/>

</ui>

Damelo con un diseño un profesional por favor

XML

<?xml version="1.0" encoding="UTF-8"?>

<ui version="4.0">

<class>GrafosWindow</class>

<widget class="QWidget" name="GrafosWindow">

<property name="geometry">

<rect>

<x>0</x>

<y>0</y>

<width>1050</width>

<height>780</height>

</rect>

</property>

<property name="windowTitle">

<string>Grafos no dirigidos</string>

</property>

<property name="styleSheet">

<string notr="true">

QWidget {

background-color: #f8fafc;

color: #1e293b;

font-family: 'Segoe UI', Arial, sans-serif;

```
}  
  
QLabel{  
    color: #334155;  
}  
  
QLabel#label_title{  
    color: #0f172a;  
    font-size: 18px;  
    font-weight: bold;  
}  
  
QLineEdit{  
    background-color: #ffffff;  
    border: 1px solid #cbd5e1;  
    border-radius: 6px;  
    padding: 6px 12px;  
    color: #0f172a;  
}  
  
QLineEdit:focus{  
    border: 2px solid #2563eb;  
}  
  
QComboBox{  
    background-color: #ffffff;  
    border: 1px solid #cbd5e1;  
    border-radius: 6px;  
    padding: 5px 10px;  
    color: #0f172a;  
}
```

```
QComboBox:focus {  
    border: 2px solid #2563eb;  
}  
  
QComboBox::drop-down {  
    subcontrol-origin: padding;  
    subcontrol-position: top right;  
    width: 25px;  
    border-left-width: 0px;  
}  
  
QPushButton {  
    background-color: #2563eb;  
    color: #ffffff;  
    border-radius: 6px;  
    padding: 7px 14px;  
    font-weight: 600;  
}  
  
QPushButton:hover {  
    background-color: #1d4ed8;  
}  
  
QPushButton:pressed {  
    background-color: #1e40af;  
}  
  
QPushButton#btnClearGraph {  
    background-color: #dc2626;  
}  
  
QPushButton#btnClearGraph:hover {
```

```
background-color: #b91c1c;  
}
```

```
QGroupBox{  
    font-weight: bold;  
    border: 1px solid #e2e8f0;  
    border-radius: 8px;  
    margin-top: 12px;  
    background-color: #ffffff;  
    padding-top: 14px;  
}
```

```
QGroupBox::title {  
    subcontrol-origin: margin;  
    subcontrol-position: top left;  
    padding: 0 6px;  
    background-color: #f8fafc;  
    color: #1e293b;  
}
```

```
QTabWidget::pane {  
    border: 1px solid #e2e8f0;  
    border-radius: 8px;  
    background-color: #ffffff;  
}
```

```
QTabBar::tab {  
    background-color: #e2e8f0;  
    color: #475569;  
    padding: 8px 16px;
```

```
border-top-left-radius: 6px;
border-top-right-radius: 6px;
margin-right: 2px;
font-weight: 600;
}
QTabBar::tab:selected {
    background-color: #2563eb;
    color: #ffffff;
}
QGraphicsView, QTableWidget, QPlainTextEdit {
    background-color: #ffffff;
    border: 1px solid #e2e8f0;
    border-radius: 6px;
    gridline-color: #f1f5f9;
    color: #0f172a;
}
QHeaderView::section {
    background-color: #f1f5f9;
    color: #334155;
    padding: 6px;
    border: none;
    border-bottom: 1px solid #cbd5e1;
    font-weight: bold;
}
</string>
</property>
```

```
<layout class="QVBoxLayout" name="verticalLayout_main">
  <property name="spacing">
    <number>16</number>
  </property>
  <property name="leftMargin">
    <number>20</number>
  </property>
  <property name="topMargin">
    <number>20</number>
  </property>
  <property name="rightMargin">
    <number>20</number>
  </property>
  <property name="bottomMargin">
    <number>20</number>
  </property>
  <item>
    <widget class="QLabel" name="label_title">
      <property name="text">
        <string>Grafos no dirigidos</string>
      </property>
    </widget>
  </item>
  <item>
    <layout class="QHBoxLayout" name="horizontalLayout_controls">
      <property name="spacing">
```

```
<number>16</number>

</property>

<item>

<widget class="QGroupBox" name="groupBox_vertices">

  <property name="title">

    <string>Vértices</string>

  </property>

  <layout class="QVBoxLayout" name="verticalLayout_vertices">

    <property name="spacing">

      <number>10</number>

    </property>

    <item>

      <widget class="QLineEdit" name="txtVertex">

        <property name="placeholderText">

          <string>Nombre del vértice</string>

        </property>

      </widget>

    </item>

    <item>

      <widget class="QPushButton" name="btnAddVertex">

        <property name="text">

          <string>Agregar vértice</string>

        </property>

      </widget>

    </item>

    <item>
```

```
<widget class="QComboBox" name="cmbVertex"/>
</item>
<item>
<widget class="QPushButton" name="btnDeleteVertex">
  <property name="text">
    <string>Eliminar vértice</string>
  </property>
</widget>
</item>
</layout>
</widget>
</item>
<item>
<widget class="QGroupBox" name="groupBox_arcos">
  <property name="title">
    <string>Arcos</string>
  </property>
  <layout class="QGridLayout" name="gridLayout_arcos">
    <property name="horizontalSpacing">
      <number>10</number>
    </property>
    <property name="verticalSpacing">
      <number>10</number>
    </property>
    <item row="0" column="0">
      <widget class="QLabel" name="label_origin">
```

```
<property name="text">
  <string>Vértice 1</string>
</property>
</widget>
</item>
<item row="0" column="1">
  <widget class="QComboBox" name="cmbOrigin"/>
</item>
<item row="1" column="0">
  <widget class="QLabel" name="label_destination">
    <property name="text">
      <string>Vértice 2</string>
    </property>
  </widget>
</item>
<item row="1" column="1">
  <widget class="QComboBox" name="cmbDestination"/>
</item>
<item row="2" column="0" colspan="2">
  <widget class="QPushButton" name="btnAddEdge">
    <property name="text">
      <string>Agregar arco</string>
    </property>
  </widget>
</item>
<item row="3" column="0" colspan="2">
```

```
<widget class="QPushButton" name="btnDeleteEdge">
  <property name="text">
    <string>Eliminar arco</string>
  </property>
</widget>
</item>
</layout>
</widget>
</item>
</layout>
</item>
<item>
  <widget class="QTabWidget" name="tabWidget_representations">
    <property name="sizePolicy">
      <sizepolicy hsizeType="Expanding" vsizeType="Expanding">
        <horstretch>0</horstretch>
        <verstretch>1</verstretch>
      </sizepolicy>
    </property>
    <property name="currentIndex">
      <number>0</number>
    </property>
    <widget class="QWidget" name="tab_visual">
      <attribute name="title">
        <string>Representación visual</string>
      </attribute>
```

```
<layout class="QVBoxLayout" name="verticalLayout_visual">
  <property name="spacing">
    <number>10</number>
  </property>
  <item>
    <widget class="QGraphicsView" name="graphicsViewGraph">
      <property name="sizePolicy">
        <sizepolicy hsize="Expanding" vsize="Expanding">
          <horstretch>0</horstretch>
          <verstretch>0</verstretch>
        </sizepolicy>
      </property>
    </widget>
  </item>
  <item>
    <widget class="QPushButton" name="btnRedrawGraph">
      <property name="text">
        <string>Redibujar grafo</string>
      </property>
    </widget>
  </item>
</layout>
</widget>
<widget class="QWidget" name="tab_matrix">
  <attribute name="title">
    <string>Matriz de adyacencia</string>
  </attribute>
</widget>
```

```
</attribute>

<layout class="QVBoxLayout" name="verticalLayout_matrix">

<item>

<widget class="QTableWidget" name="tblAdjacencyMatrix">

<property name="editTriggers">

<set>QAbstractItemView::NoEditTriggers</set>

</property>

</widget>

</item>

</layout>

</widget>

<widget class="QWidget" name="tab_adjacency_list">

<attribute name="title">

<string>Lista de adyacencia</string>

</attribute>

<layout class="QVBoxLayout" name="verticalLayout_adj_list">

<item>

<widget class="QPlainTextEdit" name="txtAdjacencyList">

<property name="readOnly">

<bool>true</bool>

</property>

</widget>

</item>

</layout>

</widget>

<widget class="QWidget" name="tab_edges_list">
```

```
<attribute name="title">
  <string>Lista de arcos</string>
</attribute>

<layout class="QVBoxLayout" name="verticalLayout_edges_list">
  <item>
    <widget class="QTableWidget" name="tblEdges">
      <property name="editTriggers">
        <set>QAbstractItemView::NoEditTriggers</set>
      </property>
      <column>
        <property name="text">
          <string>Vértice 1</string>
        </property>
      </column>
      <column>
        <property name="text">
          <string>Vértice 2</string>
        </property>
      </column>
    </widget>
  </item>
</layout>
</widget>
</widget>
</item>
<item>
```

```
<layout class="QHBoxLayout" name="horizontalLayout_footer">
```

```
<item>
```

```
<widget class="QLabel" name="lblGraphStatus">
```

```
<property name="font">
```

```
<font>
```

```
<pointsize>10</pointsize>
```

```
<bold>true</bold>
```

```
</font>
```

```
</property>
```

```
<property name="text">
```

```
<string>Vértices: 0 | Arcos: 0</string>
```

```
</property>
```

```
</widget>
```

```
</item>
```

```
<item>
```

```
<spacer name="horizontalSpacer">
```

```
<property name="orientation">
```

```
<enum>Qt::Horizontal</enum>
```

```
</property>
```

```
<property name="sizeHint" stdset="0">
```

```
<size>
```

```
<width>40</width>
```

```
<height>20</height>
```

```
</size>
```

```
</property>
```

```
</spacer>
```

```
</item>
<item>
  <widget class="QPushButton" name="btnClearGraph">
    <property name="text">
      <string>Limpiar grafo</string>
    </property>
  </widget>
</item>
</layout>
</item>
</layout>
</widget>
<resources/>
<connections/>
</ui>
```

Parte 1. Diseño de la interfaz

1. Abrir el archivo de la ventana en Qt Designer.
2. Incorporar una sección para grafos.
3. Utilizar el prompt proporcionado.
4. Agregar los componentes solicitados.
5. Asignar exactamente los objectName.
6. Aplicar layouts en todos los niveles.
7. Configurar las tablas como solo lectura.
8. Configurar el cuadro de texto como solo lectura.
9. Integrar el QGraphicsView.
10. Guardar el archivo .ui.

11. Ejecutar la aplicación.
12. Verificar que la interfaz se muestre correctamente.

Parte 2. Implementación de la estructura

13. Crear el archivo graph.py.
14. Crear la clase Graph.
15. Implementar el constructor.
16. Implementar la normalización de vértices.
17. Implementar la inserción de vértices.
18. Implementar la eliminación de vértices.
19. Implementar la inserción de arcos.
20. Implementar la eliminación de arcos.
21. Implementar los métodos de consulta.
22. Probar la clase desde consola.

Parte 3. Representaciones formales

1. Obtener la lista de adyacencia.
2. Generar la matriz de adyacencia.
3. Obtener la lista de arcos.

Universidad de León – Programación II – Ing. Alejandro Montes

Práctica de grafos no dirigidos | 21

4. Comprobar que las tres representaciones correspondan al mismo grafo.
5. Verificar que cada arco se muestre una sola vez en la lista de arcos.
6. Verificar que la matriz sea simétrica.

Parte 4. Integración con la interfaz

1. Crear una instancia de Graph.

2. Crear una instancia de QGraphicsScene.
3. Asignar la escena al QGraphicsView.
4. Configurar los componentes de solo lectura.
5. Conectar los botones.
6. Agregar vértices desde la interfaz.
7. Eliminar vértices.
8. Agregar arcos.
9. Eliminar arcos.
10. Actualizar los QComboBox.
11. Mostrar la matriz.
12. Mostrar la lista de adyacencia.
13. Mostrar la lista de arcos.
14. Dibujar los vértices.
15. Dibujar los arcos.
16. Mostrar el estado.
17. Habilitar y deshabilitar controles.
18. Implementar la opción para limpiar el grafo.
19. Implementar el botón para redibujar.

Parte 1. Diseño de la interfaz

1. Abre tu entorno de Qt Designer y carga el archivo principal de la ventana de tu aplicación (o asegúrate de incorporar el contenido de grafos.ui como una página nueva dentro de un QStackedWidget o pestaña principal).
2. Incorpora la nueva sección de grafos validando que el contenedor mantenga una jerarquía limpia y organizada.
3. Utiliza los lineamientos de diseño profesional y estructurado proporcionados en las especificaciones anteriores.

4. Asegúrate de que todos los componentes requeridos (etiquetas, campos de texto, botones, cuadros combinados, pestañas, tablas y áreas gráficas) se encuentren correctamente anidados.
5. Verifica estrictamente que los objectName coincidan en su totalidad con los solicitados (txtVertex, btnAddVertex, cmbVertex, btnDeleteVertex, cmbOrigin, cmbDestination, btnAddEdge, btnDeleteEdge, graphicsViewGraph, btnRedrawGraph, tblAdjacencyMatrix, txtAdjacencyList, tblEdges, lblGraphStatus, btnClearGraph).
6. Confirma que se hayan aplicado layouts automáticos (QVBoxLayout, QHBoxLayout, QGridLayout) en cada uno de los niveles y contenedores, evitando cualquier posicionamiento absoluto.
7. Comprueba que los componentes tabulares (tblAdjacencyMatrix, tblEdges) tengan configurada la propiedad de edición en modo exclusivo de solo lectura (NoEditTriggers).
8. Comprueba que el área de texto correspondiente (txtAdjacencyList) tenga habilitada la propiedad readOnly en true.
9. Valida que el componente graphicsViewGraph esté ubicado dentro de la primera pestaña del QTabWidget con directivas de tamaño expandible.
10. Guarda el archivo .ui actualizado (nombrándolo correctamente dentro de tu estructura de archivos, por ejemplo, grafos.ui).
11. Ejecuta el script de Python que enlaza la interfaz para poner en marcha la ventana.
12. Realiza una verificación visual comprobando que la distribución se adapte de forma fluida al redimensionar la ventana y que no existan elementos sobrepuestos.

Parte 2. Implementación de la estructura

13. Crea un nuevo archivo dentro de tu estructura modular de paquetes (por ejemplo, en la ruta correspondiente a estructuras no lineales o principales) con el nombre graph.py.
14. Declara dentro del archivo la clase principal encargada de la lógica matemática y estructural: class Graph:.

15. Implementa el método constructor (`__init__`) para inicializar las estructuras de almacenamiento internas (como un diccionario de adyacencia donde las llaves sean los vértices y los valores sean conjuntos o listas de vecinos).
16. Implementa un método interno de normalización de vértices (para asegurar consistencia en mayúsculas, eliminación de espacios en blanco o validación de entradas vacías).
17. Implementa el método de inserción de vértices, verificando que no existan duplicados y actualizando la estructura interna.
18. Implementa el método de eliminación de vértices, asegurándote de remover también todas las incidencias o arcos conectados a dicho vértice en los demás nodos del grafo.
19. Implementa el método de inserción de arcos para grafos **no dirigidos** (añadiendo la conexión bidireccional entre el vértice de origen y el de destino, y validando que ambos vértices existan).
20. Implementa el método de eliminación de arcos (removiendo la relación recíproca entre ambos vértices de forma segura).
21. Implementa métodos de consulta básicos (como obtener la lista de vértices, los vecinos de un nodo específico, el grado de un vértice o el conteo total de elementos).
22. Realiza pruebas preliminares instanciando y ejecutando la clase directamente desde la consola de Python para validar el comportamiento lógico antes de conectarla a la interfaz gráfica.

Parte 3. Representaciones formales

1. Implementa el método para obtener y formatear la **lista de adyacencia** como una estructura legible que mapee cada vértice con sus respectivos vecinos conectados.
2. Implementa el método para generar la **matriz de adyacencia** estructurada en forma de tabla bidimensional (con ceros y unos según la existencia de conexiones).
3. Implementa el método para extraer la **lista de arcos** del grafo.

4. Comprueba mediante pruebas lógicas que las tres representaciones (lista de adyacencia, matriz y lista de arcos) reflejen exactamente el mismo estado del grafo.
5. Verifica que en la lista de arcos cada conexión no dirigida se muestre una sola vez (evitando duplicados inversos como (A, B) y (B, C) redundantes).
6. Verifica que la matriz de adyacencia generada cumpla estrictamente con la propiedad de ser **simétrica** respecto a su diagonal principal (debido a la naturaleza no dirigida del grafo).

Parte 4. Integración con la interfaz

1. Crea una instancia de tu clase Graph dentro del controlador principal de la interfaz gráfica.
2. Crea una instancia de QGraphicsScene para gestionar los elementos visuales del grafo.
3. Asigna dicha escena al componente visual mediante `self.graphicsViewGraph.setScene(scene)`.
4. Configura programáticamente las restricciones de solo lectura para los componentes que lo requieran si no se definieron estáticamente en el archivo `.ui`.
5. Conecta las señales de los botones (clicked) y eventos de cambio con los métodos correspondientes del controlador (`btnAddVertex`, `btnDeleteVertex`, `btnAddEdge`, `btnDeleteEdge`, `btnClearGraph`, `btnRedrawGraph`).
6. Programa la lógica para agregar vértices desde el campo `txtVertex` interactuando con la clase Graph.
7. Programa la lógica para eliminar el vértice seleccionado en el combo de control.
8. Programa la inserción de arcos utilizando los valores seleccionados en `cmbOrigin` y `cmbDestination`.
9. Programa la eliminación de arcos de forma coherente con la selección de la interfaz.
10. Actualiza dinámicamente los elementos de los componentes `QComboBox` (`cmbVertex`, `cmbOrigin`, `cmbDestination`) cada vez que se agreguen o eliminen vértices.

11. Actualiza y despliega la matriz de adyacencia en el componente tabular `tblAdjacencyMatrix`.
12. Muestra la lista de adyacencia formateada dentro del cuadro de texto multilínea `txtAdjacencyList`.
13. Muestra la lista de arcos actualizada en la tabla correspondiente `tblEdges`.
14. Implementa el algoritmo de renderizado geométrico para **dibujar los vértices** en forma de nodos circulares distribuidos dentro de la escena gráfica.
15. Implementa el algoritmo para **dibujar los arcos** mediante líneas conectoras directas entre las coordenadas calculadas de los vértices.
16. Actualiza de forma automática la etiqueta de estado `lblGraphStatus` reflejando el conteo actual de vértices y arcos (Vértices: X | Arcos: Y).
17. Implementa reglas de validación para habilitar o deshabilitar controles dinámicamente (por ejemplo, deshabilitar botones de arcos si no hay suficientes vértices disponibles).
18. Implementa la funcionalidad del botón `btnClearGraph` para vaciar por completo el grafo, limpiar las tablas, el área de texto y reiniciar la escena gráfica.
19. Implementa la funcionalidad del botón `btnRedrawGraph` para recalcular posiciones y refrescar la representación visual en pantalla a petición del usuario.

Dame los codigos por favor de cada cosa

A continuación tienes el código completo y modularizado para implementar la estructura del grafo y el controlador en Python que da vida a la interfaz `grafos.ui` que diseñamos.

1. Lógica de la estructura: `graph.py`

Guarda este archivo dentro de tu paquete de estructuras (por ejemplo, en `estructuras/no_lineales/graph.py`). Contiene la lógica matemática y de representación para un **grafo no dirigido**.

Python

```
class Graph:
```

```
    def __init__(self):
```

```
# Diccionario para almacenar el grafo: {vertice: {vecino1, vecino2, ...}}
```

```
self.adj_list = {}
```

```
def normalizar_vertice(self, vertex):
```

```
    """Limpia espacios y estandariza el nombre del vértice."""
```

```
    if not vertex:
```

```
        return ""
```

```
    return str(vertex).strip()
```

```
def agregar_vertice(self, vertex):
```

```
    v = self.normalizar_vertice(vertex)
```

```
    if not v:
```

```
        raise ValueError("El nombre del vértice no puede estar vacío.")
```

```
    if v in self.adj_list:
```

```
        raise ValueError(f"El vértice '{v}' ya existe en el grafo.")
```

```
    self.adj_list[v] = set()
```

```
    return True
```

```
def eliminar_vertice(self, vertex):
```

```
    v = self.normalizar_vertice(vertex)
```

```
    if v not in self.adj_list:
```

```
        raise ValueError(f"El vértice '{v}' no existe en el grafo.")
```

```
# Eliminar todas las aristas conectadas a este vértice en los demás nodos
```

```
for vecino in list(self.adj_list[v]):
```

```
self.adj_list[vecino].discard(v)
```

```
# Eliminar el vértice principal
```

```
del self.adj_list[v]
```

```
return True
```

```
def agregar_arco(self, v1, v2):
```

```
    vertice1 = self.normalizar_vertice(v1)
```

```
    vertice2 = self.normalizar_vertice(v2)
```

```
    if not vertice1 or not vertice2:
```

```
        raise ValueError("Los vértices de origen y destino son obligatorios.")
```

```
    if vertice1 not in self.adj_list or vertice2 not in self.adj_list:
```

```
        raise ValueError("Ambos vértices deben existir en el grafo.")
```

```
    if vertice1 == vertice2:
```

```
        raise ValueError("No se permiten lazos (arcos hacia el mismo vértice).")
```

```
# Al ser un grafo no dirigido, se conecta en ambas direcciones
```

```
self.adj_list[vertice1].add(vertice2)
```

```
self.adj_list[vertice2].add(vertice1)
```

```
return True
```

```
def eliminar_arco(self, v1, v2):
```

```
    vertice1 = self.normalizar_vertice(v1)
```

```
    vertice2 = self.normalizar_vertice(v2)
```

```
if vertice1 in self.adj_list and vertice2 in self.adj_list:
```

```
    self.adj_list[vertice1].discard(vertice2)
```

```
    self.adj_list[vertice2].discard(vertice1)
```

```
    return True
```

```
return False
```

```
def limpiar(self):
```

```
    self.adj_list.clear()
```

```
# Métodos de consulta y representaciones formales
```

```
def obtener_vertices(self):
```

```
    return sorted(list(self.adj_list.keys()))
```

```
def obtener_arcos(self):
```

```
    """Retorna una lista única de arcos sin duplicados inversos."""
```

```
    arcos = set()
```

```
    for u in self.adj_list:
```

```
        for v in self.adj_list[u]:
```

```
            # Ordenar la tupla asegura que (A, B) y (B, A) se traten como el mismo arco
```

```
            arco = tuple(sorted((u, v)))
```

```
            arcos.add(arco)
```

```
    return sorted(list(arcos))
```

```
def obtener_lista_adyacencia_texto(self):
```

```
    texto = []
```

```
    for v in self.obtener_vertices():
```

```

        vecinos = ", ".join(sorted(list(self.adj_list[v])))

        texto.append(f"{v} -> [ {vecinos} ]")

    return "\n".join(texto)

```

```

def obtener_matriz_adyacencia(self):

```

```

    vertices = self.obtener_vertices()

```

```

    n = len(vertices)

```

```

    matriz = [[0] * n for _ in range(n)]

```

```

    vertice_a_indice = {v: i for i, v in enumerate(vertices)}

```

```

    for u in vertices:

```

```

        for v in self.adj_list[u]:

```

```

            i = vertice_a_indice[u]

```

```

            j = vertice_a_indice[v]

```

```

            matriz[i][j] = 1

```

```

            matriz[j][i] = 1 # Simetría garantizada

```

```

    return vertices, matriz

```

```

def contar_vertices(self):

```

```

    return len(self.adj_list)

```

```

def contar_arcos(self):

```

```

    return len(self.obtener_arcos())

```

2. Controlador de la Interfaz: main_graph_controller.py

Crea este archivo en la raíz de tu proyecto para enlazar la interfaz grafos.ui con la clase Graph, encargándose de las tablas, la actualización de combos y el renderizado visual automático en el QGraphicsView.

Python

```
import sys
```

```
import math
```

```
from PyQt5.QtWidgets import QApplication, QWidget, QMessageBox,  
QTableWidgetItem
```

```
from PyQt5.uic import loadUi
```

```
from PyQt5.QtGui import QPen, QBrush, QFont
```

```
from PyQt5.QtCore import Qt
```

```
from estructuras.no_lineales.graph import Graph
```

```
class GraphWindow(QWidget):
```

```
    def __init__(self):
```

```
        super().__init__()
```

```
        # Carga el archivo .ui generado anteriormente
```

```
        loadUi("grafos.ui", self)
```

```
        self.grafo = Graph()
```

```
        # Configurar la escena gráfica para el QGraphicsView
```

```
        from PyQt5.QtWidgets import QGraphicsScene
```

```
        self.scene = QGraphicsScene()
```

```
        self.graphicsViewGraph.setScene(self.scene)
```

```
# Conexiones de botones con sus respectivas funciones

self.btnAddVertex.clicked.connect(self.agregar_vertice)

self.btnDeleteVertex.clicked.connect(self.eliminar_vertice)

self.btnAddEdge.clicked.connect(self.agregar_arco)

self.btnDeleteEdge.clicked.connect(self.eliminar_arco)

self.btnClearGraph.clicked.connect(self.limpiar_grafo)

self.btnRedrawGraph.clicked.connect(self.actualizar_interfaz)
```

```
def agregar_vertice(self):

    nombre = self.txtVertex.text()

    try:

        self.grafo.agregar_vertice(nombre)

        self.txtVertex.clear()

        self.actualizar_interfaz()

    except Exception as e:

        QMessageBox.warning(self, "Advertencia", str(e))
```

```
def eliminar_vertice(self):

    nombre = self.cmbVertex.currentText()

    if not nombre:

        return

    try:

        self.grafo.eliminar_vertice(nombre)

        self.actualizar_interfaz()

    except Exception as e:

        QMessageBox.warning(self, "Error", str(e))
```

```

def agregar_arco(self):

    v1 = self.cmbOrigin.currentText()

    v2 = self.cmbDestination.currentText()

    try:

        self.grafo.agregar_arco(v1, v2)

        self.actualizar_interfaz()

    except Exception as e:

        QMessageBox.warning(self, "Advertencia", str(e))


def eliminar_arco(self):

    v1 = self.cmbOrigin.currentText()

    v2 = self.cmbDestination.currentText()

    try:

        if not self.grafo.eliminar_arco(v1, v2):

            QMessageBox.information(self, "Información", "El arco seleccionado no
existe.")

            self.actualizar_interfaz()

    except Exception as e:

        QMessageBox.warning(self, "Error", str(e))


def limpiar_grafo(self):

    self.grafo.limpiar()

    self.actualizar_interfaz()


def actualizar_interfaz(self):

```

1. Actualizar ComboBoxes

```
vertices = self.grafo.obtener_vertices()
```

```
self.cmbVertex.clear()
```

```
self.cmbVertex.addItems(vertices)
```

```
self.cmbOrigin.clear()
```

```
self.cmbOrigin.addItems(vertices)
```

```
self.cmbDestination.clear()
```

```
self.cmbDestination.addItems(vertices)
```

2. Actualizar Etiqueta de Estado

```
num_v = self.grafo.contar_vertices()
```

```
num_a = self.grafo.contar_arcos()
```

```
self.lblGraphStatus.setText(f"Vértices: {num_v} | Arcos: {num_a}")
```

3. Actualizar Lista de Adyacencia (Plain Text Edit)

```
self.txtAdjacencyList.setPlainText(self.grafo.obtener_lista_adyacencia_texto())
```

4. Actualizar Matriz de Adyacencia (Table Widget)

```
_, matriz = self.grafo.obtener_matriz_adyacencia()
```

```
n = len(vertices)
```

```
self.tblAdjacencyMatrix.setRowCount(n)
```

```
self.tblAdjacencyMatrix.setColumnCount(n)
```

```
self.tblAdjacencyMatrix.setHorizontalHeaderLabels(vertices)
```

```
self.tblAdjacencyMatrix.setVerticalHeaderLabels(vertices)
```

```
for i in range(n):
```

```
    for j in range(n):
```

```
        item = QTableWidgetItem(str(matriz[i][j]))
```

```
        item.setTextAlignment(Qt.AlignCenter)
```

```
        self.tblAdjacencyMatrix.setItem(i, j, item)
```

```
# 5. Actualizar Lista de Arcos (Table Widget)
```

```
arcos = self.grafo.obtener_arcos()
```

```
self.tblEdges.setRowCount(len(arcos))
```

```
self.tblEdges.setColumnCount(2)
```

```
self.tblEdges.setHorizontalHeaderLabels(["Vértice 1", "Vértice 2"])
```

```
for row, (u, v) in enumerate(arcos):
```

```
    item1 = QTableWidgetItem(u)
```

```
    item2 = QTableWidgetItem(v)
```

```
    item1.setTextAlignment(Qt.AlignCenter)
```

```
    item2.setTextAlignment(Qt.AlignCenter)
```

```
    self.tblEdges.setItem(row, 0, item1)
```

```
    self.tblEdges.setItem(row, 1, item2)
```

```
# 6. Dibujar el Grafo Visualmente en el QGraphicsView
```

```
self.dibujar_grafo()
```

```
def dibujar_grafo(self):
```

```

self.scene.clear()

vertices = self.grafo.obtener_vertices()

n = len(vertices)

if n == 0:

    return


# Calcular dimensiones del área de dibujo

radio_nodo = 22

ancho_canvas = max(400, self.graphicsViewGraph.width() - 50)
alto_canvas = max(300, self.graphicsViewGraph.height() - 50)

centro_x = ancho_canvas / 2
centro_y = alto_canvas / 2

radio_circulo = min(centro_x, centro_y) - 60

if radio_circulo < 50:

    radio_circulo = 120


# Asignar coordenadas (X, Y) distribuidas en forma circular para los vértices
posiciones = {}

for i, v in enumerate(vertices):

    angulo = 2 * math.pi * i / n

    x = centro_x + radio_circulo * math.cos(angulo)

    y = centro_y + radio_circulo * math.sin(angulo)

    posiciones[v] = (x, y)


# Dibujar líneas de arcos primero para que queden por detrás de los nodos
pen_arco = QPen(Qt.darkGray, 2)

```

```

arcos_dibujados = set()

for u in self.grafo.adj_list:
    for v in self.grafo.adj_list[u]:
        if (v, u) not in arcos_dibujados:
            arcos_dibujados.add((u, v))
            x1, y1 = posiciones[u]
            x2, y2 = posiciones[v]
            self.scene.addLine(x1, y1, x2, y2, pen_arco)

# Dibujar nodos (círculos y texto)
brush_nodo = QBrush(Qt.white)
pen_nodo = QPen(Qt.blue, 2)
font_nodo = QFont("Segoe UI", 10, QFont.Bold)

for v, (x, y) in posiciones.items():
    self.scene.addEllipse(x - radio_nodo, y - radio_nodo, radio_nodo * 2, radio_nodo
* 2, pen_nodo, brush_nodo)
    text_item = self.scene.addText(str(v), font_nodo)
    rect = text_item.boundingRect()
    text_item.setPos(x - rect.width() / 2, y - rect.height() / 2)

if __name__ == "__main__":
    app = QApplication(sys.argv)
    ventana = GraphWindow()
    ventana.show()

```

```
sys.exit(app.exec_())
```

Ahora, dame el archivo load de prafo por favor

Supongo que quisiste decir el archivo **.ui** de grafos que acabamos de diseñar y estilizar con calidad profesional.

El código completo y listo para guardarse como **grafos.ui** es el siguiente:

XML

```
<?xml version="1.0" encoding="UTF-8"?>

<ui version="4.0">

<class>GrafosWindow</class>

<widget class="QWidget" name="GrafosWindow">

<property name="geometry">

<rect>

<x>0</x>

<y>0</y>

<width>1050</width>

<height>780</height>

</rect>

</property>

<property name="windowTitle">

<string>Grafos no dirigidos</string>

</property>

<property name="styleSheet">

<string notr="true">

QWidget {

    background-color: #f8fafc;

    color: #1e293b;
```

```
    font-family: 'Segoe UI', Arial, sans-serif;
}

QLabel{
    color: #334155;
}

QLabel#label_title{
    color: #0f172a;
    font-size: 18px;
    font-weight: bold;
}

QLineEdit{
    background-color: #ffffff;
    border: 1px solid #cbd5e1;
    border-radius: 6px;
    padding: 6px 12px;
    color: #0f172a;
}

QLineEdit:focus{
    border: 2px solid #2563eb;
}

QComboBox{
    background-color: #ffffff;
    border: 1px solid #cbd5e1;
    border-radius: 6px;
    padding: 5px 10px;
    color: #0f172a;
```

```
}  
  
QComboBox:focus {  
    border: 2px solid #2563eb;  
}  
  
QComboBox::drop-down {  
    subcontrol-origin: padding;  
    subcontrol-position: top right;  
    width: 25px;  
    border-left-width: 0px;  
}  
  
QPushButton {  
    background-color: #2563eb;  
    color: #ffffff;  
    border-radius: 6px;  
    padding: 7px 14px;  
    font-weight: 600;  
}  
  
QPushButton:hover {  
    background-color: #1d4ed8;  
}  
  
QPushButton:pressed {  
    background-color: #1e40af;  
}  
  
QPushButton#btnClearGraph {  
    background-color: #dc2626;  
}
```

```
QPushButton#btnClearGraph:hover {  
    background-color: #b91c1c;  
}
```

```
QGroupBox {  
    font-weight: bold;  
    border: 1px solid #e2e8f0;  
    border-radius: 8px;  
    margin-top: 12px;  
    background-color: #ffffff;  
    padding-top: 14px;  
}
```

```
QGroupBox::title {  
    subcontrol-origin: margin;  
    subcontrol-position: top left;  
    padding: 0 6px;  
    background-color: #f8fafc;  
    color: #1e293b;  
}
```

```
QTabWidget::pane {  
    border: 1px solid #e2e8f0;  
    border-radius: 8px;  
    background-color: #ffffff;  
}
```

```
QTabBar::tab {  
    background-color: #e2e8f0;  
    color: #475569;
```

```
padding: 8px 16px;
border-top-left-radius: 6px;
border-top-right-radius: 6px;
margin-right: 2px;
font-weight: 600;
}
QTabBar::tab:selected {
    background-color: #2563eb;
    color: #ffffff;
}
QGraphicsView, QTableWidget, QPlainTextEdit {
    background-color: #ffffff;
    border: 1px solid #e2e8f0;
    border-radius: 6px;
    gridline-color: #f1f5f9;
    color: #0f172a;
}
QHeaderView::section {
    background-color: #f1f5f9;
    color: #334155;
    padding: 6px;
    border: none;
    border-bottom: 1px solid #cbd5e1;
    font-weight: bold;
}
</string>
```

```
</property>
<layout class="QVBoxLayout" name="verticalLayout_main">
  <property name="spacing">
    <number>16</number>
  </property>
  <property name="leftMargin">
    <number>20</number>
  </property>
  <property name="topMargin">
    <number>20</number>
  </property>
  <property name="rightMargin">
    <number>20</number>
  </property>
  <property name="bottomMargin">
    <number>20</number>
  </property>
  <item>
    <widget class="QLabel" name="label_title">
      <property name="text">
        <string>Grafos no dirigidos</string>
      </property>
    </widget>
  </item>
  <item>
    <layout class="QHBoxLayout" name="horizontalLayout_controls">
```

```
<property name="spacing">
  <number>16</number>
</property>
<item>
  <widget class="QGroupBox" name="groupBox_vertices">
    <property name="title">
      <string>Vértices</string>
    </property>
    <layout class="QVBoxLayout" name="verticalLayout_vertices">
      <property name="spacing">
        <number>10</number>
      </property>
      <item>
        <widget class="QLineEdit" name="txtVertex">
          <property name="placeholderText">
            <string>Nombre del vértice</string>
          </property>
        </widget>
      </item>
      <item>
        <widget class="QPushButton" name="btnAddVertex">
          <property name="text">
            <string>Agregar vértice</string>
          </property>
        </widget>
      </item>
    </layout>
  </widget>
</item>
```

```
<item>

  <widget class="QComboBox" name="cmbVertex"/>

</item>

<item>

  <widget class="QPushButton" name="btnDeleteVertex">

    <property name="text">

      <string>Eliminar vértice</string>

    </property>

  </widget>

</item>

</layout>

</widget>

</item>

<item>

  <widget class="QGroupBox" name="groupBox_arcos">

    <property name="title">

      <string>Arcos</string>

    </property>

    <layout class="QGridLayout" name="gridLayout_arcos">

      <property name="horizontalSpacing">

        <number>10</number>

      </property>

      <property name="verticalSpacing">

        <number>10</number>

      </property>

      <item row="0" column="0">
```

```
<widget class="QLabel" name="label_origin">
    <property name="text">
        <string>Vértice 1</string>
    </property>
</widget>
</item>
<item row="0" column="1">
    <widget class="QComboBox" name="cmbOrigin"/>
</item>
<item row="1" column="0">
    <widget class="QLabel" name="label_destination">
        <property name="text">
            <string>Vértice 2</string>
        </property>
    </widget>
</item>
<item row="1" column="1">
    <widget class="QComboBox" name="cmbDestination"/>
</item>
<item row="2" column="0" colspan="2">
    <widget class="QPushButton" name="btnAddEdge">
        <property name="text">
            <string>Agregar arco</string>
        </property>
    </widget>
</item>
```

```
<item row="3" column="0" colspan="2">
  <widget class="QPushButton" name="btnDeleteEdge">
    <property name="text">
      <string>Eliminar arco</string>
    </property>
  </widget>
</item>
</layout>
</widget>
</item>
</layout>
</item>
<item>
  <widget class="QTabWidget" name="tabWidget_representations">
    <property name="sizePolicy">
      <sizepolicy hstyp="Expanding" vstyp="Expanding">
        <horstretch>0</horstretch>
        <verstretch>1</verstretch>
      </sizepolicy>
    </property>
    <property name="currentIndex">
      <number>0</number>
    </property>
    <widget class="QWidget" name="tab_visual">
      <attribute name="title">
        <string>Representación visual</string>
      </attribute>
    </widget>
  </widget>
</item>
```

```
</attribute>

<layout class="QVBoxLayout" name="verticalLayout_visual">

  <property name="spacing">

    <number>10</number>

  </property>

  <item>

    <widget class="QGraphicsView" name="graphicsViewGraph">

      <property name="sizePolicy">

        <sizepolicy hsize="Expanding" vsize="Expanding">

          <horstretch>0</horstretch>

          <verstretch>0</verstretch>

        </sizepolicy>

      </property>

    </widget>

  </item>

  <item>

    <widget class="QPushButton" name="btnRedrawGraph">

      <property name="text">

        <string>Redibujar grafo</string>

      </property>

    </widget>

  </item>

</layout>

</widget>

<widget class="QWidget" name="tab_matrix">

  <attribute name="title">
```

```
<string>Matriz de adyacencia</string>
</attribute>
<layout class="QVBoxLayout" name="verticalLayout_matrix">
  <item>
    <widget class="QTableWidget" name="tblAdjacencyMatrix">
      <property name="editTriggers">
        <set>QAbstractItemView::NoEditTriggers</set>
      </property>
    </widget>
  </item>
</layout>
</widget>
<widget class="QWidget" name="tab_adjacency_list">
  <attribute name="title">
    <string>Lista de adyacencia</string>
  </attribute>
  <layout class="QVBoxLayout" name="verticalLayout_adj_list">
    <item>
      <widget class="QPlainTextEdit" name="txtAdjacencyList">
        <property name="readOnly">
          <bool>true</bool>
        </property>
      </widget>
    </item>
  </layout>
</widget>
```

```
<widget class="QWidget" name="tab_edges_list">
  <attribute name="title">
    <string>Lista de arcos</string>
  </attribute>
  <layout class="QVBoxLayout" name="verticalLayout_edges_list">
    <item>
      <widget class="QTableWidget" name="tblEdges">
        <property name="editTriggers">
          <set>QAbstractItemView::NoEditTriggers</set>
        </property>
        <column>
          <property name="text">
            <string>Vértice 1</string>
          </property>
        </column>
        <column>
          <property name="text">
            <string>Vértice 2</string>
          </property>
        </column>
      </widget>
    </item>
  </layout>
</widget>
</widget>
</item>
```

```
<item>

<layout class="QHBoxLayout" name="horizontalLayout_footer">

<item>

<widget class="QLabel" name="lblGraphStatus">

  <property name="font">

    <font>

      <pointsize>10</pointsize>

      <bold>true</bold>

    </font>

  </property>

  <property name="text">

    <string>Vértices: 0 | Arcos: 0</string>

  </property>

</widget>

</item>

<item>

<spacer name="horizontalSpacer">

  <property name="orientation">

    <enum>Qt::Horizontal</enum>

  </property>

  <property name="sizeHint" stdset="0">

    <size>

      <width>40</width>

      <height>20</height>

    </size>

  </property>

</item>
```

```

</spacer>
</item>
<item>
<widget class="QPushButton" name="btnClearGraph">
<property name="text">
<string>Limpiar grafo</string>
</property>
</widget>
</item>
</layout>
</item>
</layout>
</widget>
<resources/>
<connections/>
</ui>

```

el load es el archivo en donde voy a enlazar todo

Ah, te referías al script de Python que carga la interfaz gráfica y enlaza todo el funcionamiento con la clase Graph. Ese es el archivo controlador que te compartí anteriormente (main_graph_controller.py).

Aquí lo tienes listo para que lo guardes (por ejemplo, como main_graphs.py o app.py) asegurándote de que la ruta de importación de tu clase Graph coincida exactamente con la estructura de carpetas de tu proyecto:

Python

```
import sys
```

```
import math
```

```
from PyQt5.QtWidgets import QApplication, QWidget, QMessageBox,
QTableWidgetItem
```

```
from PyQt5.uic import loadUi

from PyQt5.QtGui import QPen, QBrush, QFont

from PyQt5.QtCore import Qt


# Asegúrate de que esta ruta coincida con donde guardaste graph.py
from estructuras.no_lineales.graph import Graph


class GraphWindow(QWidget):

    def __init__(self):
        super().__init__()

        # Carga el archivo .ui de grafos que diseñamos
        loadUi("grafos.ui", self)

        # Instancia de la estructura formal del grafo
        self.grafo = Graph()

        # Configurar la escena gráfica para el QGraphicsView
        from PyQt5.QtWidgets import QGraphicsScene
        self.scene = QGraphicsScene()
        self.graphicsViewGraph.setScene(self.scene)

        # Conexiones de botones con sus respectivas funciones lógicas
        self.btnAddVertex.clicked.connect(self.agregar_vertice)
        self.btnDeleteVertex.clicked.connect(self.eliminar_vertice)
        self.btnAddEdge.clicked.connect(self.agregar_arco)
        self.btnDeleteEdge.clicked.connect(self.eliminar_arco)
```

```
self.btnClearGraph.clicked.connect(self.limpiar_grafo)

self.btnRedrawGraph.clicked.connect(self.actualizar_interfaz)
```

```
def agregar_vertice(self):

    nombre = self.txtVertex.text()

    try:

        self.grafo.agregar_vertice(nombre)

        self.txtVertex.clear()

        self.actualizar_interfaz()

    except Exception as e:

        QMessageBox.warning(self, "Advertencia", str(e))
```

```
def eliminar_vertice(self):

    nombre = self.cmbVertex.currentText()

    if not nombre:

        return

    try:

        self.grafo.eliminar_vertice(nombre)

        self.actualizar_interfaz()

    except Exception as e:

        QMessageBox.warning(self, "Error", str(e))
```

```
def agregar_arco(self):

    v1 = self.cmbOrigin.currentText()

    v2 = self.cmbDestination.currentText()

    try:
```

```

        self.grafo.agregar_arco(v1, v2)

        self.actualizar_interfaz()

    except Exception as e:

        QMessageBox.warning(self, "Advertencia", str(e))


def eliminar_arco(self):

    v1 = self.cmbOrigin.currentText()

    v2 = self.cmbDestination.currentText()

    try:

        if not self.grafo.eliminar_arco(v1, v2):

            QMessageBox.information(self, "Información", "El arco seleccionado no
existe.")

            self.actualizar_interfaz()

    except Exception as e:

        QMessageBox.warning(self, "Error", str(e))


def limpiar_grafo(self):

    self.grafo.limpiar()

    self.actualizar_interfaz()


def actualizar_interfaz(self):

    # 1. Actualizar ComboBoxes con la lista de vértices actual

    vertices = self.grafo.obtener_vertices()

    self.cmbVertex.clear()

    self.cmbVertex.addItems(vertices)

```

```
self.cmbOrigin.clear()
```

```
self.cmbOrigin.addItems(vertices)
```

```
self.cmbDestination.clear()
```

```
self.cmbDestination.addItems(vertices)
```

```
# 2. Actualizar Etiqueta de Estado inferior
```

```
num_v = self.grafo.contar_vertices()
```

```
num_a = self.grafo.contar_arcos()
```

```
self.lblGraphStatus.setText(f"Vértices: {num_v} | Arcos: {num_a}")
```

```
# 3. Actualizar la pestaña "Lista de adyacencia"
```

```
self.txtAdjacencyList.setPlainText(self.grafo.obtener_lista_adyacencia_texto())
```

```
# 4. Actualizar la pestaña "Matriz de adyacencia"
```

```
_, matriz = self.grafo.obtener_matriz_adyacencia()
```

```
n = len(vertices)
```

```
self.tblAdjacencyMatrix.setRowCount(n)
```

```
self.tblAdjacencyMatrix.setColumnCount(n)
```

```
self.tblAdjacencyMatrix.setHorizontalHeaderLabels(vertices)
```

```
self.tblAdjacencyMatrix.setVerticalHeaderLabels(vertices)
```

```
for i in range(n):
```

```
    for j in range(n):
```

```
        item = QTableWidgetItem(str(matriz[i][j]))
```

```
item.setTextAlignment(Qt.AlignCenter)

self.tblAdjacencyMatrix.setItem(i, j, item)
```

5. Actualizar la pestaña "Lista de arcos"

```
arcos = self.grafo.obtener_arcos()

self.tblEdges.setRowCount(len(arcos))

self.tblEdges.setColumnCount(2)

self.tblEdges.setHorizontalHeaderLabels(["Vértice 1", "Vértice 2"])
```

```
for row, (u, v) in enumerate(arcos):

    item1 = QTableWidgetItem(u)

    item2 = QTableWidgetItem(v)

    item1.setTextAlignment(Qt.AlignCenter)

    item2.setTextAlignment(Qt.AlignCenter)

    self.tblEdges.setItem(row, 0, item1)

    self.tblEdges.setItem(row, 1, item2)
```

6. Renderizar gráficamente el grafo en la pestaña "Representación visual"

```
self.dibujar_grafo()
```

```
def dibujar_grafo(self):

    self.scene.clear()

    vertices = self.grafo.obtener_vertices()

    n = len(vertices)

    if n == 0:

        return
```

```

# Calcular dimensiones relativas para distribuir los nodos de forma circular

radio_nodo = 24

ancho_canvas = max(400, self.graphicsViewGraph.width() - 50)
alto_canvas = max(300, self.graphicsViewGraph.height() - 50)

centro_x = ancho_canvas / 2
centro_y = alto_canvas / 2

radio_circulo = min(centro_x, centro_y) - 70

if radio_circulo < 60:
    radio_circulo = 130

posiciones = {}

for i, v in enumerate(vertices):
    angulo = 2 * math.pi * i / n
    x = centro_x + radio_circulo * math.cos(angulo)
    y = centro_y + radio_circulo * math.sin(angulo)
    posiciones[v] = (x, y)

# Dibujar líneas de arcos primero para que queden por debajo de los nodos

pen_arco = QPen(Qt.darkGray, 2)
arcos_dibujados = set()

for u in self.grafo.adj_list:
    for v in self.grafo.adj_list[u]:
        if (v, u) not in arcos_dibujados:
            arcos_dibujados.add((u, v))
            x1, y1 = posiciones[u]

```

```

        x2, y2 = posiciones[v]

        self.scene.addLine(x1, y1, x2, y2, pen_arco)

# Dibujar los nodos en pantalla (círculos y etiquetas de texto)
brush_nodo = QBrush(Qt.white)
pen_nodo = QPen(Qt.blue, 2)
font_nodo = QFont("Segoe UI", 10, QFont.Bold)

for v, (x, y) in posiciones.items():
    self.scene.addEllipse(x - radio_nodo, y - radio_nodo, radio_nodo * 2, radio_nodo
* 2, pen_nodo, brush_nodo)

    text_item = self.scene.addText(str(v), font_nodo)

    rect = text_item.boundingRect()

    text_item.setPos(x - rect.width() / 2, y - rect.height() / 2)

if __name__ == "__main__":
    app = QApplication(sys.argv)

    ventana = GraphWindow()

    ventana.show()

    sys.exit(app.exec_())

```

en donde aparece eliminar arco, me debe de eliminar todo lo que tenga ahí, y en redibujar al presionarlo me debe de volver a aparecer el grafo que se eliminó previamente

Para lograr lo que pides, vamos a modificar dos cosas:

1. **Eliminar arco / Borrar todo:** Si te refieres a que el botón "**Eliminar arco**" vacíe todo el grafo (o si prefieres que limpie los campos), podemos ajustar su comportamiento o utilizar un respaldo temporal para guardar el estado del

grafo antes de limpiarlo. Sin embargo, para cumplir exactamente con que "al presionar redibujar me vuelva a aparecer el grafo que se eliminó previamente", implementaremos un mecanismo de **papelera/respaldo** con el botón de limpiar o redibujar.

2. **Restaurar con "Redibujar grafo"**: Guardaremos una copia del estado actual del grafo antes de borrarlo o limpiarlo, de modo que al presionar **"Redibujar grafo"** se recupere la información eliminada.

Aquí tienes los códigos actualizados para graph.py y tu controlador, añadiendo esta funcionalidad de respaldo y restauración:

1. Actualización en graph.py

Añadiremos un método de respaldo temporal o copia de seguridad en la estructura:

Python

```
class Graph:
```

```
    def __init__(self):
```

```
        self.adj_list = {}
```

```
        self._backup_adj_list = {} # Respaldo para restaurar
```

```
    def normalizar_vertice(self, vertex):
```

```
        if not vertex:
```

```
            return ""
```

```
        return str(vertex).strip()
```

```
    def agregar_vertice(self, vertex):
```

```
        v = self.normalizar_vertice(vertex)
```

```
        if not v:
```

```
            raise ValueError("El nombre del vértice no puede estar vacío.")
```

```
        if v in self.adj_list:
```

```
            raise ValueError(f"El vértice '{v}' ya existe en el grafo.")
```

```
self.adj_list[v] = set()
```

```
return True
```

```
def eliminar_vertice(self, vertex):
```

```
    v = self.normalizar_vertice(vertex)
```

```
    if v not in self.adj_list:
```

```
        raise ValueError(f"El vértice '{v}' no existe en el grafo.")
```

```
    for vecino in list(self.adj_list[v]):
```

```
        self.adj_list[vecino].discard(v)
```

```
    del self.adj_list[v]
```

```
    return True
```

```
def agregar_arco(self, v1, v2):
```

```
    vertice1 = self.normalizar_vertice(v1)
```

```
    vertice2 = self.normalizar_vertice(v2)
```

```
    if not vertice1 or not vertice2:
```

```
        raise ValueError("Los vértices son obligatorios.")
```

```
    if vertice1 not in self.adj_list or vertice2 not in self.adj_list:
```

```
        raise ValueError("Ambos vértices deben existir.")
```

```
    if vertice1 == vertice2:
```

```
        raise ValueError("No se permiten lazos.")
```

```
    self.adj_list[vertice1].add(vertice2)
```

```
    self.adj_list[vertice2].add(vertice1)
```

```
    return True
```

```
def eliminar_arco(self, v1, v2):
```

```
vertice1 = self.normalizar_vertice(v1)
vertice2 = self.normalizar_vertice(v2)
if vertice1 in self.adj_list and vertice2 in self.adj_list:
    self.adj_list[vertice1].discard(vertice2)
    self.adj_list[vertice2].discard(vertice1)
    return True
return False
```

```
def respaldar_y_limpiar(self):
    """Guarda una copia exacta antes de limpiar para poder restaurarla."""
    self._backup_adj_list = {k: set(v) for k, v in self.adj_list.items()}
    self.adj_list.clear()
```

```
def restaurar_grafo(self):
    """Restaura el grafo desde el último respaldo guardado."""
    if self._backup_adj_list:
        self.adj_list = {k: set(v) for k, v in self._backup_adj_list.items()}
        return True
    return False
```

```
def limpiar(self):
    self._backup_adj_list = {k: set(v) for k, v in self.adj_list.items()}
    self.adj_list.clear()
```

```
def obtener_vertices(self):
    return sorted(list(self.adj_list.keys()))
```

```
def obtener_arcos(self):
```

```
    arcos = set()
```

```
    for u in self.adj_list:
```

```
        for v in self.adj_list[u]:
```

```
            arcos.add(tuple(sorted((u, v))))
```

```
    return sorted(list(arcos))
```

```
def obtener_lista_adyacencia_texto(self):
```

```
    texto = []
```

```
    for v in self.obtener_vertices():
```

```
        vecinos = ", ".join(sorted(list(self.adj_list[v])))
```

```
        texto.append(f"{v} -> [ {vecinos} ]")
```

```
    return "\n".join(texto)
```

```
def obtener_matriz_adyacencia(self):
```

```
    vertices = self.obtener_vertices()
```

```
    n = len(vertices)
```

```
    matriz = [[0] * n for _ in range(n)]
```

```
    vertice_a_indice = {v: i for i, v in enumerate(vertices)}
```

```
    for u in vertices:
```

```
        for v in self.adj_list[u]:
```

```
            i = vertice_a_indice[u]
```

```
            j = vertice_a_indice[v]
```

```
            matriz[i][j] = 1
```

```
            matriz[j][i] = 1
```

```
return vertices, matriz
```

```
def contar_vertices(self):
```

```
    return len(self.adj_list)
```

```
def contar_arcos(self):
```

```
    return len(self.obtener_arcos())
```

2. Actualización en el archivo de carga (main_graphs.py)

Modificaremos la acción del botón de eliminar arco (para que actúe como borrado general o limpieza si así lo requieres) y haremos que el botón **"Redibujar grafo"** recupere el grafo previamente eliminado:

Python

```
import sys
```

```
import math
```

```
from PyQt5.QtWidgets import QApplication, QWidget, QMessageBox,  
QTableWidgetItem, QGraphicsScene
```

```
from PyQt5.uic import loadUi
```

```
from PyQt5.QtGui import QPen, QBrush, QFont
```

```
from PyQt5.QtCore import Qt
```

```
from estructuras.no_lineales.graph import Graph
```

```
class GraphWindow(QWidget):
```

```
    def __init__(self):
```

```
        super().__init__()
```

```
        loadUi("grafos.ui", self)
```

```
self.grafo = Graph()

self.scene = QGraphicsScene()

self.graphicsViewGraph.setScene(self.scene)


# Conexiones

self.btnAddVertex.clicked.connect(self.agregar_vertice)

self.btnDeleteVertex.clicked.connect(self.eliminar_vertice)

self.btnAddEdge.clicked.connect(self.agregar_arco)


# Modificamos "Eliminar arco" para que limpie/elimine todo el contenido actual
self.btnDeleteEdge.clicked.connect(self.eliminar_todo_por_completo)


self.btnClearGraph.clicked.connect(self.limpiar_grafo)


# El botón de redibujar ahora también restaurará el grafo previo si fue borrado
self.btnRedrawGraph.clicked.connect(self.restaurar_y_redibujar)


def agregar_vertice(self):

    nombre = self.txtVertex.text()

    try:

        self.grafo.agregar_vertice(nombre)

        self.txtVertex.clear()

        self.actualizar_interfaz()

    except Exception as e:

        QMessageBox.warning(self, "Advertencia", str(e))
```

```

def eliminar_vertice(self):

    nombre = self.cmbVertex.currentText()

    if not nombre:

        return

    try:

        self.grafo.eliminar_vertice(nombre)

        self.actualizar_interfaz()

    except Exception as e:

        QMessageBox.warning(self, "Error", str(e))


def agregar_arco(self):

    v1 = self.cmbOrigin.currentText()

    v2 = self.cmbDestination.currentText()

    try:

        self.grafo.agregar_arco(v1, v2)

        self.actualizar_interfaz()

    except Exception as e:

        QMessageBox.warning(self, "Advertencia", str(e))


def eliminar_todo_por_completo(self):

    """Elimina/limpia todo lo que se tenga en el grafo y lo respalda."""

    self.grafo.respalda_y_limpiar()

    self.actualizar_interfaz()

    QMessageBox.information(self, "Información", "Se ha eliminado todo el
contenido. Puedes recuperarlo haciendo clic en 'Redibujar grafo'.")

```

```

def limpiar_grafo(self):

    self.grafo.limpiar()

    self.actualizar_interfaz()


def restaurar_y_redibujar(self):

    """Restaura el grafo eliminado previamente al presionar Redibujar"""

    restaurado = self.grafo.restaurar_grafo()

    self.actualizar_interfaz()

    if restaurado:

        QMessageBox.information(self, "Restaurado", "¡Se ha recuperado el grafo previo exitosamente!")


def actualizar_interfaz(self):

    vertices = self.grafo.obtener_vertices()


    self.cmbVertex.clear()

    self.cmbVertex.addItems(vertices)

    self.cmbOrigin.clear()

    self.cmbOrigin.addItems(vertices)

    self.cmbDestination.clear()

    self.cmbDestination.addItems(vertices)


    num_v = self.grafo.contar_vertices()

    num_a = self.grafo.contar_arcos()

    self.lblGraphStatus.setText(f"Vértices: {num_v} | Arcos: {num_a}")

```

```
self.txtAdjacencyList.setPlainText(self.grafo.obtener_lista_adyacencia_texto())
```

```
_, matriz = self.grafo.obtener_matriz_adyacencia()
```

```
n = len(vertices)
```

```
self.tblAdjacencyMatrix.setRowCount(n)
```

```
self.tblAdjacencyMatrix.setColumnCount(n)
```

```
self.tblAdjacencyMatrix.setHorizontalHeaderLabels(vertices)
```

```
self.tblAdjacencyMatrix.setVerticalHeaderLabels(vertices)
```

```
for i in range(n):
```

```
    for j in range(n):
```

```
        item = QTableWidgetItem(str(matriz[i][j]))
```

```
        item.setTextAlignment(Qt.AlignCenter)
```

```
        self.tblAdjacencyMatrix.setItem(i, j, item)
```

```
arcos = self.grafo.obtener_arcos()
```

```
self.tblEdges.setRowCount(len(arcos))
```

```
self.tblEdges.setColumnCount(2)
```

```
self.tblEdges.setHorizontalHeaderLabels(["Vértice 1", "Vértice 2"])
```

```
for row, (u, v) in enumerate(arcos):
```

```
    item1 = QTableWidgetItem(u)
```

```
    item2 = QTableWidgetItem(v)
```

```
    item1.setTextAlignment(Qt.AlignCenter)
```

```
    item2.setTextAlignment(Qt.AlignCenter)
```

```
    self.tblEdges.setItem(row, 0, item1)
```

```
self.tblEdges.setItem(row, 1, item2)
```

```
self.dibujar_grafo()
```

```
def dibujar_grafo(self):
```

```
    self.scene.clear()
```

```
    vertices = self.grafo.obtener_vertices()
```

```
    n = len(vertices)
```

```
    if n == 0:
```

```
        return
```

```
    radio_nodo = 24
```

```
    ancho_canvas = max(400, self.graphicsViewGraph.width() - 50)
```

```
    alto_canvas = max(300, self.graphicsViewGraph.height() - 50)
```

```
    centro_x = ancho_canvas / 2
```

```
    centro_y = alto_canvas / 2
```

```
    radio_circulo = min(centro_x, centro_y) - 70
```

```
    if radio_circulo < 60:
```

```
        radio_circulo = 130
```

```
    posiciones = {}
```

```
    for i, v in enumerate(vertices):
```

```
        angulo = 2 * math.pi * i / n
```

```
        x = centro_x + radio_circulo * math.cos(angulo)
```

```
        y = centro_y + radio_circulo * math.sin(angulo)
```

```
        posiciones[v] = (x, y)
```

```

pen_arco = QPen(Qt.darkGray, 2)

arcos_dibujados = set()

for u in self.grafo.adj_list:
    for v in self.grafo.adj_list[u]:
        if (v, u) not in arcos_dibujados:
            arcos_dibujados.add((u, v))

            x1, y1 = posiciones[u]
            x2, y2 = posiciones[v]

            self.scene.addLine(x1, y1, x2, y2, pen_arco)

brush_nodo = QBrush(Qt.white)
pen_nodo = QPen(Qt.blue, 2)
font_nodo = QFont("Segoe UI", 10, QFont.Bold)

for v, (x, y) in posiciones.items():
    self.scene.addEllipse(x - radio_nodo, y - radio_nodo, radio_nodo * 2, radio_nodo
* 2, pen_nodo, brush_nodo)

    text_item = self.scene.addText(str(v), font_nodo)

    rect = text_item.boundingRect()

    text_item.setPos(x - rect.width() / 2, y - rect.height() / 2)

if __name__ == "__main__":
    app = QApplication(sys.argv)

    ventana = GraphWindow()

```

```
ventana.show()
```

```
sys.exit(app.exec_())
```